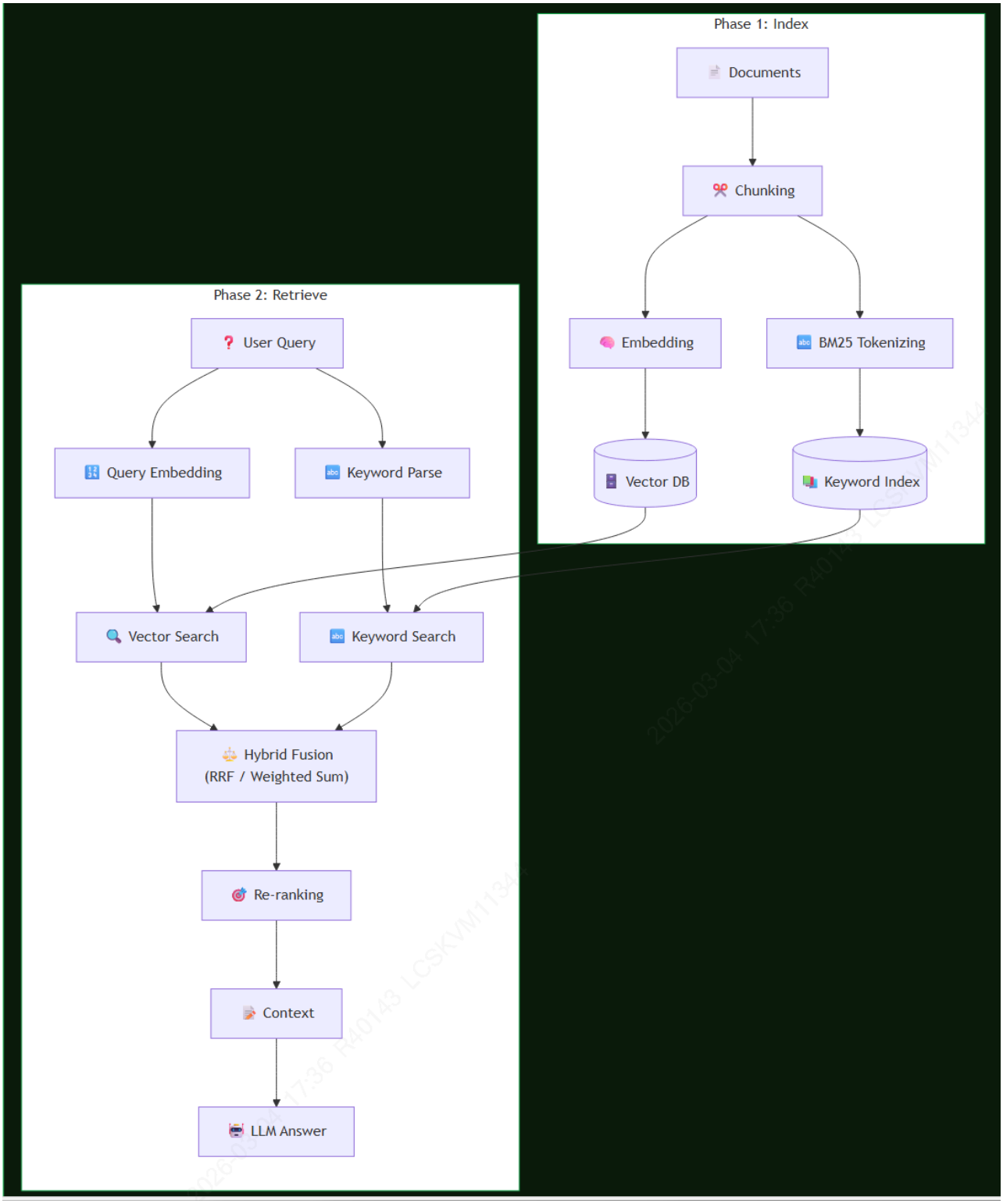
2.1.3.3 Hybrid Search in RAG Pipelines

Vietnam Build Center

Exported on 03/05/2026

Table of Contents

0.1 Which Database Supports 4
0.2 — Challenges with Hybrid Search 4
0.2.1 ⚠️ Challenge 1: Dual Index Synchronization 4
0.2.2 ✂️ Challenge 2: Chunking Strategy Affects Both Indexes Differently 4
0.2.3 📏 Challenge 3: Score Normalization Before Fusion 5
0.2.4 🎛️ Challenge 4: Fusion Weight Tuning (α) 5
0.2.5 ⏱️ Challenge 5: Latency — Two Searches in Parallel 6
1 🏁 Summary 8


Phase 1: Index
Documents
Chunking
Embedding
BM25 Tokenizing
Vector DB
Keyword Index
Phase 2: Retrieve
User Query
Query Embedding
Keyword Parse
Vector Search
Keyword Search
Hybrid Fusion
(RRF / Weighted Sum)
Re-ranking
Context
LLM Answer

0.1 Which Database Supports

Different databases handle **dense** (vector search) and **sparse** (keyword search) search differently:

- **Dense** = Vector search (floating-point embeddings, like [0.1, 0.8, -0.3, ...])
- **Sparse** = Keyword/BM25 search (word counts, like {"python": 3, "tutorial": 1})

When to use which:

Situation	Recommended DB	Reason
Already using Postgres	**pgvector**	Minimal change, SQL stays the same
Log search + AI features	**Elasticsearch**	BM25 is battle-tested, vector is added on
New AI-native RAG app	**Weaviate** or **Qdrant**	Hybrid search first-class feature
Fully managed, no ops	**Pinecone**	Serverless, scales automatically
Billions of vectors	**Milvus**	Built for massive scale

0.2 — Challenges with Hybrid Search

0.2.1 ⚠ Challenge 1: Dual Index Synchronization

> *"When a document updates, BOTH indexes must update — or they drift out of sync."*

Example: A user asks "What's the return policy?" The vector search finds the updated policy, but keyword search finds the *6-month-old* version. The hybrid system merges them — and the old doc contaminates the answer.

Fix: Use a write pipeline (e.g., Kafka) that writes to *both* indexes in a single transaction, or treat one as the source of truth and sync the other.

0.2.2 ✂ Challenge 2: Chunking Strategy Affects Both Indexes Differently

> *"A chunk that's great for vector search might be terrible for keyword search — and vice versa."*

- **Vector search prefers:** Larger chunks (~512 tokens) with rich context — more meaning per chunk

- **Keyword search prefers:** Smaller chunks (~128 tokens) — specific keywords aren't diluted by surrounding text

Example:

Chunk A (large, 512 tokens): *"Our company was founded in 1998... [300 words]... The return policy allows 30 days..."*

- **Vector**: ✓ Good — captures overall document meaning
- **Keyword**: ✗ Bad — searching "return policy" ranks this lower because the term density is low

Fix: Consider dual-chunking — large chunks for vector index, smaller chunks for keyword index, with shared document IDs to link them.

0.2.3 📏 Challenge 3: Score Normalization Before Fusion

> *"Comparing apples to oranges: BM25 scores can be 0–100, cosine scores are 0–1."*

You *must* normalize before combining. Two common approaches:

Method	Formula	Pro	Con
Min-Max	`(score - min) / (max - min)`	Simple	Sensitive to outliers
Z-score	`(score - mean) / std_dev`	Handles outliers	Needs enough data

Example: BM25 returns `[92.1, 45.3, 12.8]` and cosine returns `[0.95, 0.71, 0.43]`. Without normalizing, BM25 will dominate every fusion calculation just because its numbers are bigger.

0.2.4 🎛️ Challenge 4: Fusion Weight Tuning (α)

> *"How much should I trust keyword vs vector? There's no universal answer."*

The parameter α (alpha) controls the balance:

- `α = 1.0` → 100% vector search (ignore keywords)
- `α = 0.0` → 100% keyword search (ignore vectors)
- `α = 0.5` → Equal mix

Example tuning by domain:

Domain	Recommended α	Reason
Legal document search	0.2	Exact terms matter most; "Section 12(b)(3)" must match exactly
Customer support chatbot	0.7	Intent matters more than exact words
Code search	0.3	Function names, error codes = exact
Creative writing assistant	0.8	Meaning and tone matter more than keywords
Medical records	0.5	Balance: ICD codes (exact) + symptoms (semantic)

Fix: Use offline evaluation with labeled queries and A/B testing to find your domain's sweet spot for α.

0.2.5 ⏱️ Challenge 5: Latency — Two Searches in Parallel

> *"Two searches take longer than one — even run in parallel, there's overhead."*

Latency breakdown for a typical hybrid query:

Step	Time	Notes
Keyword search (BM25)	~15–25ms	Very fast, well-optimized
Vector embedding	~10–30ms	Depends on model size
ANN vector search	~10–20ms	Runs after embedding
Fusion (RRF)	~2–5ms	Pure computation, negligible
Re-ranking (optional)	~50–200ms	The expensive step!
Total (no re-rank)	**~40–60ms**	Acceptable
Total (with re-rank)	**~100–300ms**	Noticeable delay

Fixes:

- Cache frequent query embeddings
- Skip re-ranking for simple/navigational queries
- Use a lighter cross-encoder model for re-ranking
- Set a timeout: if re-ranking exceeds 150ms, skip it and use fused results directly

1 🏁 Summary

> ***Bottom line:*** *Neither keyword nor vector search is perfect alone. Hybrid search is the industry standard for production-grade AI search — combining the precision of keywords with the intelligence of vectors. Start with* ***RRF fusion****, use* ***Weaviate or Qdrant*** *if starting fresh, and tune your* ***α parameter*** *with real user queries.*